

Plan de Pruebas — Sistema de Tickets e Incidentes

Sistema de Tickets e Incidentes — Entrega # 2 · Plan de Pruebas

Universidad Galileo · Postgrado en Diseño y Desarrollo de Software · Infraestructura en la Nube Ciclo Mayo–Junio 2026

Equipo:

- Luis André Morales
 - Erick Estuardo Saban
-

0. Qué cambió desde la entrega anterior

Esta es la **primera versión del Plan de Pruebas**. Toma como insumo la Entrega #1 (docs/E1_TicketSystem.md), específicamente:

- Los 8 casos de uso priorizados (CU-01 a CU-08) y sus criterios de éxito (§3 de E1).
- Las funcionalidades específicas §4.1 a §4.7 (creación con severidad/tipo, priorización automática, escalamiento por SLA, historial inmutable, adjuntos en S3, notificaciones asíncronas, RBAC).
- El mapeo a servicios AWS de §5 (ECS Fargate, RDS PostgreSQL, S3, SQS + workers, VPC, JWT del IdP, observabilidad).
- El alcance dentro/fuera del scope (§6 de E1) y las preguntas abiertas que aún están sin cerrar (conurrencia, IdP concreto, motor de email, mecanismo del job de escalamiento, etc.).
- Los 8 mockups low-fi en docs/mockups/ para las pruebas de UX.

Este plan **no modifica el alcance** definido en E1; lo cubre. Las decisiones aún abiertas en §6 de E1 se reflejan aquí como **supuestos explícitos** o como **riesgos** (ver §14).

Esta versión del plan se **alineó 1:1 con los 8 temas explícitos solicitados por la rúbrica del curso** (objetivos, alcance, cronograma, tipos de prueba, entornos, riesgos, criterios de aceptación con entrada y salida, entregables). En concreto se reforzó §2.2 Ambientes con una tabla detallada por entorno (ENT-001 a ENT-005), se reforzó §15 Cronograma con una tabla maestra de fases (CRON-F1 a CRON-F12) y un Gantt en ASCII, se reescribió §13 separando explícitamente **criterios de entrada (entry criteria)** por nivel (EC-IN-) y **criterios de salida (exit criteria)** por nivel (EC-OUT-), y se agregó la nueva sección §17 **Entregables del plan de pruebas** (ENT-DEL-001 a ENT-DEL-020).

1. Objetivos y alcance del plan de pruebas

1.1 Objetivos

1. Verificar que el Sistema de Tickets e Incidentes cumple los criterios de éxito declarados para cada caso de uso CU-01 a CU-08.
2. Validar que las funcionalidades específicas §4.1 a §4.7 se comportan correctamente bajo casos felices, errores controlados, escenarios límite y condiciones de carga realista.
3. Asegurar la integridad y la inmutabilidad del historial de eventos (§4.4), pieza central de la trazabilidad del sistema.
4. Garantizar que el control de acceso por rol (§4.7) se aplica consistentemente en cada endpoint y que no existen rutas de escalada de privilegios.
5. Demostrar que el sistema sostiene la carga esperada con latencia P95 < 2 s en endpoints críticos (CU-06) y que degrada de forma controlada bajo estrés.
6. Detectar vulnerabilidades del OWASP Top 10 relevantes (injection, broken auth, IDOR, SSRF en adjuntos) antes de la entrega final.
7. Evaluar la usabilidad de los 8 mockups frente a las heurísticas de Nielsen y los criterios básicos de WCAG 2.1 AA.

1.2 Alcance — qué se prueba

Categoría	Alcance	Referencia E1
Lógica de dominio backend	Cálculo de prioridad, reglas de SLA, transiciones de estado, RBAC, generación de URLs prefirmadas	§4.1, §4.2, §4.3, §4.5, §4.7
API REST	POST /tickets , PATCH /tickets/{id}/state , POST /tickets/{id}/assign , GET /tickets , endpoints de comentarios, adjuntos e historial	§5 Cómputo
Persistencia	Tablas tickets , ticket_events , sla_rules y su consistencia transaccional	§5 Base de datos
Procesamiento asíncrono	Publicación y consumo de mensajes en SQS, worker de notificaciones, job de escalamiento	§5 Procesamiento asíncrono, §4.3, §4.6
Almacenamiento de archivos	Subida a S3, expiración de URLs prefirmadas, separación de metadatos	§4.5
Seguridad	Validación de JWT, RBAC por endpoint, headers de seguridad, validación de inputs	§4.7, §5 Seguridad
Rendimiento	Endpoints críticos bajo carga y estrés, comportamiento del worker	§5 Observabilidad
Experiencia de usuario	8 mockups low-fi (docs/mockups/)	§7 de E1

1.3 Fuera del alcance del plan

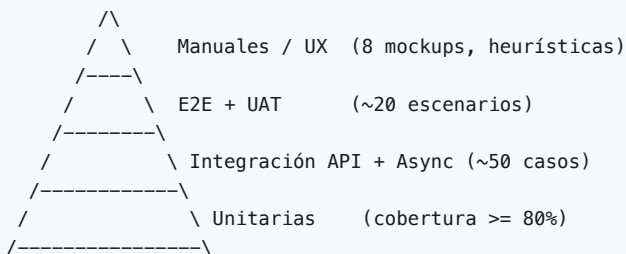
Coherente con §6 de E1, **no se prueba**:

- Aplicación móvil nativa, dashboards en tiempo real con WebSockets, integraciones con PagerDuty/Datadog/Jira/ServiceNow.
- Multi-tenant, internacionalización, federación multi-IdP, chatbot/IA embebida, motor de workflow configurable, knowledge base, encuestas CSAT/NPS.
- Frontend productivo implementado en código (solo se realizan pruebas heurísticas y de accesibilidad sobre los mockups low-fi).
- Pruebas de migración desde sistemas externos preexistentes (sin precedente declarado).
- Pruebas de localización idiomática (el sistema vive solo en español).

2. Estrategia general y niveles de prueba

2.1 Pirámide de pruebas

El plan se apoya en la pirámide clásica de Mike Cohn, adaptada al stack:



La base son **pruebas unitarias** rápidas y deterministas sobre la lógica de dominio. El segundo nivel cubre la **integración** del API con RDS, SQS y S3 — preferentemente con servicios reales (LocalStack o testcontainers en CI; entornos efímeros AWS en staging). Encima se posan los flujos **E2E** que recorren un caso de uso completo, y arriba quedan las pruebas **manuales** de UX y aceptación con el profesor/cliente.

2.2 Ambientes

Esta sección detalla los entornos sobre los que se ejecutan los distintos tipos de prueba. Cada ambiente tiene un propósito, un conjunto de componentes desplegados, una política de datos, un mecanismo de acceso y una política de refresco bien definidos. Los IDs `ENT-001` a `ENT-005` se usan en la sección §13 (entry criteria) para referirse a precondiciones de ambiente.

ID	Ambiente	Propósito	Componentes desplegados	Datos	Acceso	Refresh policy	Tipos de prueba ejecutados
ENT-001	local/dev developer	Loop de desarrollo individual: ejecución de unitarias y exploración manual	docker-compose con Postgres, LocalStack (S3/SQS/SES), API y worker locales	Fixtures en memoria; seeds estáticos del repo	Solo el desarrollador en su máquina	A demanda (docker compose down -v)	TU (unitarias), exploratorias
ENT-002	CI ephemeral	Pipeline por PR: unitarias + integración + análisis estático	Testcontainers (Postgres) y LocalStack levantados por job; imagen Docker construida del SHA del PR	Esquema limpio por job; factorías con Faker	Tokens efímeros del runner de GitHub Actions; sin acceso humano	Se destruye al final de cada job	TU, TI (subset), análisis estático (tfLint, tfsec, checkov, gitLeaks, Trivy)
ENT-003	dev shared	Integración continua de la rama main : validación de cambios integrados entre módulos	EKS dev (1 AZ), RDS Single-AZ, SQS, S3 dev, SES sandbox, Cognito dev pool	Dataset sintético sembrado nightly (~1k tickets)	SSO del equipo + IAM IRSA por workload	Re-seed nightly; recreación completa con Terraform semanal	TI completas, TE2E nightly, smoke post-deploy
ENT-004	staging	Validación pre-producción: UAT, carga, estrés, seguridad, UX guiada	EKS multi-AZ idéntico a prod (escala menor), RDS Multi-AZ, SQS+DLQ, S3, SES sandbox, Cognito, WAF, API Gateway	Dataset realista anonimizado (10k–50k tickets); refrescable bajo demanda	SSO del equipo + acceso de lectura para profesor/cliente	Snapshot semanal de RDS; re-seed bajo demanda antes de carga/estrés; ventanas dedicadas	TE2E, TUAT, TSEC (ZAP/Burp), TC (carga), TS (estrés), TUX guiada
ENT-005	prod	Operación real con usuarios finales	EKS multi-AZ, RDS Multi-AZ con PITR, SQS+DLQ, S3 con object lock, SES productivo, Cognito prod, WAF, CloudFront, X-Ray	Datos productivos reales	SSO con MFA + aprobaciones por cambio; break-glass auditado	Sin re-seed; backups automáticos diarios + PITR 7 días	Smoke tests post-deploy únicamente; observabilidad continua

Notas operativas de ambientes:

- Ningún ambiente comparte VPC, base de datos ni bucket S3 con otro (RNF-07).
- LocalStack se usa solo en ENT-001 y ENT-002 ; a partir de dev shared siempre se usan servicios AWS reales para evitar divergencia mock/real.
- Las pruebas de carga (TC) y estrés (TS) **solo** se ejecutan en ENT-004 staging dentro de ventanas previamente anunciadas al equipo, para no contaminar métricas de otros tipos de prueba ni incurrir en costos no planificados.
- Las pruebas de seguridad invasivas (ZAP active scan, sqlmap, Burp) **nunca** se ejecutan contra ENT-005 prod ; se limitan a ENT-004 .

2.3 Trazabilidad

Cada caso de prueba lleva un ID con prefijo según su tipo (TU-, TI-, TE2E-, TUAT-, TSEC-, TC-, TS-, TUX-) y referencia explícita al CU y a la sección §4.x que cubre. La matriz consolidada vive en §12.

2.4 Definition of Ready y Definition of Done

- Ready para ejecutar:** el caso tiene precondiciones reproducibles, datos preparados y criterios de aprobación medibles.
- Done de un nivel:** 100% de los casos P0/P1 ejecutados, defectos críticos resueltos o aceptados con justificación, métricas reportadas.

3. Requerimientos de aplicación cloud

Esta sección documenta los **requerimientos de infraestructura y servicios cloud** sobre los que se desplegará y operará el Sistema de Tickets e Incidentes. Toda la infraestructura se levanta en **AWS** y es **100% automatizada con Terraform** (provisión, configuración, actualización y destrucción), siguiendo el principio de Infrastructure as Code (IaC) y manteniendo el árbol del repositorio `infra/` como única fuente de verdad.

Estos requerimientos extienden y concretan el §5 (Mapeo a conceptos del curso) de E1, sustituyendo la opción inicial de **ECS Fargate** por una arquitectura basada en **Amazon EKS** con contenedores orquestados por Kubernetes, alineada con la decisión técnica del equipo para esta entrega.

3.1 Principios de arquitectura cloud

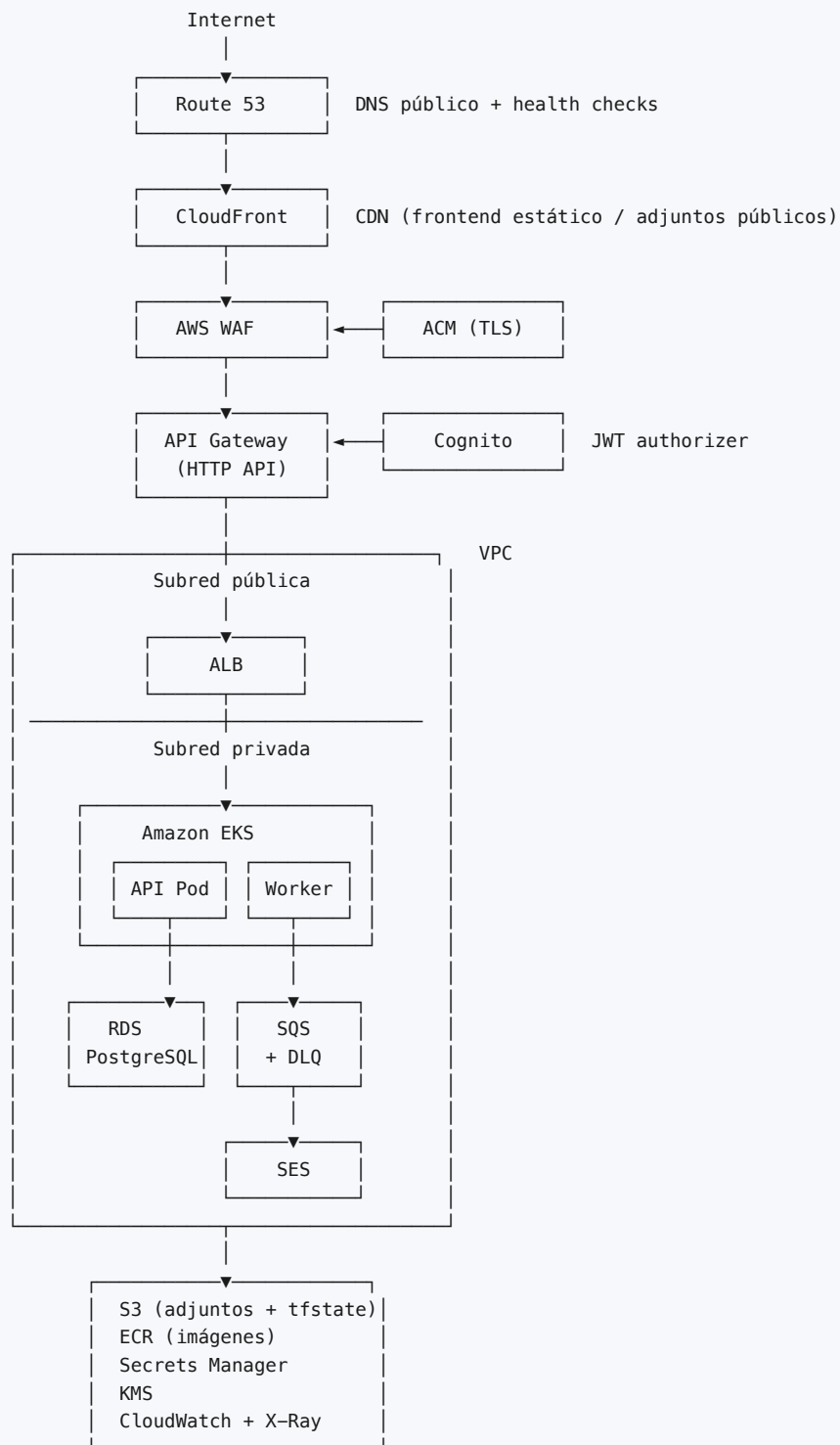
1. **Infrastructure as Code (IaC) con Terraform** — Ningún recurso se crea desde la consola AWS; todo cambio pasa por código revisado y aplicado por pipeline.
2. **Inmutabilidad** — Las imágenes de contenedor son inmutables (tag por SHA del commit); los nodos del cluster se reemplazan, no se parchean en caliente.
3. **Multi-ambiente** — Configuración separada por entorno (`infra/envs/dev` , `infra/envs/prod`) reutilizando los módulos base.
4. **Mínimo privilegio** — Roles IAM granulares por workload mediante IRSA (IAM Roles for Service Accounts).
5. **Defensa en profundidad** — Red privada para datos y workloads, exposición controlada en la capa pública (API Gateway + ALB).
6. **Trazabilidad y observabilidad por defecto** — Logs, métricas y trazas habilitados desde el día uno (CloudWatch + contenedores instrumentados).

3.2 Servicios AWS requeridos

#	Servicio AWS	Propósito en el sistema	Provisión
1	Amazon EKS (Elastic Kubernetes Service)	Orquestador de contenedores que ejecuta la API REST, los workers asíncronos y el job de escalamiento por SLA. Cluster gestionado con node groups en subredes privadas.	Terraform (módulo <code>eks</code>)
2	Amazon ECR (Elastic Container Registry)	Image Registry privado que almacena las imágenes Docker de la API y los workers, versionadas por SHA y escaneadas en push (Trivy / ECR scan).	Terraform
3	Amazon API Gateway (HTTP API)	Punto de entrada público al backend. Termina TLS, aplica throttling, valida JWT contra el IdP y enruta hacia el ALB interno del cluster EKS.	Terraform
4	Amazon Route 53	DNS público para el dominio del sistema (<code>tickets.example.com</code> , <code>api.tickets.example.com</code>). Health checks sobre los endpoints expuestos.	Terraform
5	Amazon S3	Almacenamiento de adjuntos de los tickets (§4.5 de E1) con URLs prefirmadas y expiración. Adicionalmente se usa como backend de estado de Terraform (bucket dedicado con versionado).	Terraform
6	Amazon RDS (PostgreSQL)	Base de datos relacional para <code>tickets</code> , <code>ticket_events</code> , <code>sla_rules</code> y <code>user_preferences</code> . Multi-AZ en producción, Single-AZ en dev.	Terraform (módulo <code>rds</code>)
7	Amazon SQS	Colas de mensajes para notificaciones asíncronas (email/Slack) y disparadores del job de escalamiento (§4.6, §4.3 de E1). Una cola por flujo más DLQ asociada.	Terraform
8	Amazon SES	Motor de envío de email para las notificaciones asíncronas (cierra la pregunta abierta de E1 §6 sobre el motor de email).	Terraform
9	AWS Certificate Manager (ACM)	Emisión y renovación automática de certificados TLS para los dominios públicos servidos vía API Gateway y CloudFront.	Terraform
10	AWS WAF	Reglas de protección sobre el API Gateway: rate limiting por IP, bloqueo de bots conocidos, OWASP managed rules.	Terraform
11	Amazon CloudFront	CDN delante del frontend estático (cuando se construya en entregas futuras) y para servir los adjuntos públicos (cuando aplique) a través de Origin Access Control sobre S3.	Terraform
12	AWS IAM	Roles y políticas por workload. Uso de IRSA (IAM Roles for Service Accounts) en EKS para que cada Pod asuma exclusivamente los permisos que necesita (acceso a SQS, S3, RDS Data API o Secrets Manager).	Terraform

#	Servicio AWS	Propósito en el sistema	Provisión
13	AWS Secrets Manager	Almacén de secretos: credenciales de RDS, claves de firma JWT de servicio a servicio, tokens de integración con Slack. Rotación automática habilitada.	Terraform
14	AWS KMS	Llaves de cifrado para RDS (at-rest), S3 (SSE-KMS para adjuntos), EBS de nodos EKS y secretos de Secrets Manager.	Terraform
15	Amazon VPC	Red privada con subredes públicas (NAT/ALB) y privadas (EKS workers, RDS, Endpoints). Endpoints VPC para S3 y ECR para evitar salida por Internet.	Terraform
16	Amazon CloudWatch	Logs, métricas y alarmas (§5 Observabilidad de E1): SLA cumplidos, escalamientos, latencia P95 API, errores del worker, profundidad de colas SQS.	Terraform
17	AWS X-Ray	Trazas distribuidas API Gateway → EKS → RDS/SQS/S3 para diagnóstico de latencia P95/P99.	Terraform
18	Amazon EventBridge Scheduler	Programador del job periódico que revisa tickets con SLA vencido y dispara escalamiento (§4.3, CU-04 de E1; cierra la pregunta abierta sobre el mecanismo del job).	Terraform
19	Amazon Cognito	IdP gestionado para emisión de JWT (cierra la pregunta abierta de E1 §6 sobre el IdP). Integrado con API Gateway como authorizer.	Terraform
20	Amazon DynamoDB	Backend de bloqueo de estado de Terraform (<code>tfstate-lock</code>) para evitar aplicaciones concurrentes corruptas.	Terraform

3.3 Topología de despliegue



3.4 Automatización con Terraform

Toda la infraestructura se gestiona desde el directorio `infra/` del repositorio, siguiendo la estructura ya iniciada en E1:

```

infra/
├─ main.tf          # Composición de módulos (network, eks, rds, sqs, s3, etc.)
├─ provider.tf      # Configuración de provider AWS y backend remoto
├─ variables.tf     # Variables compartidas
├─ outputs.tf       # Outputs (cluster endpoint, dns name, etc.)
├─ envs/
│   └─ dev/dev.tfvars    # Valores por ambiente
│   └─ prod/prod.tfvars
└─ modules/          # Módulos reutilizables (network, eks, ecr, api-gw, etc.)

```

Aspecto	Decisión
Versión de Terraform	<code>>= 1.7.0</code> (lock con <code>.terraform-lock.hcl</code> versionado)
Provider AWS	<code>hashicorp/aws ~> 5.x</code>
Backend de estado	S3 con versionado + DynamoDB para locking
Estructura	Módulos reutilizables + workspaces por ambiente
Pipeline	GitHub Actions: <code>terraform fmt</code> , <code>validate</code> , <code>plan</code> en PR, <code>apply</code> tras merge a <code>main</code> con aprobación manual para <code>prod</code>
Validación estática	<code>tflint</code> , <code>terraform fmt -check</code> , <code>tfsec</code> y <code>checkov</code> corren en CI
Inmutabilidad	Imágenes Docker etiquetadas por SHA del commit (no <code>latest</code>); Terraform referencia el tag exacto
Secretos	Nunca en <code>.tfvars</code> versionados; se inyectan vía AWS Secrets Manager o variables de entorno del runner

3.5 Requerimientos no funcionales sobre la infraestructura

RNF	Requerimiento	Cómo lo cumple la arquitectura
RNF-01 Disponibilidad	99.5% mensual en <code>prod</code>	EKS multi-AZ, RDS Multi-AZ, API Gateway regional, SQS gestionado
RNF-02 Escalabilidad horizontal	API debe escalar de 2 a 20 réplicas según carga	HPA (Horizontal Pod Autoscaler) en EKS basado en CPU y RPS
RNF-03 Latencia API	P95 < 500 ms para endpoints de lectura, < 800 ms para escritura	EKS dimensionado, RDS con índices, X-Ray para detectar cuellos
RNF-04 RPO / RTO	RPO ≤ 24h, RTO ≤ 4h	Backups automáticos de RDS (snapshots diarios + PITR 7 días); IaC permite re-crear todo en otra región si fuera necesario
RNF-05 Seguridad de red	RDS y SQS jamás accesibles desde Internet	Security groups: solo desde Pods de EKS; subred privada sin ruta a IGW
RNF-06 Cifrado	At-rest y in-transit por defecto	KMS para EBS, RDS, S3, Secrets Manager; TLS 1.2+ en ACM/API Gateway
RNF-07 Aislamiento por ambiente	<code>dev</code> y <code>prod</code> no comparten datos ni recursos	Cuentas AWS separadas o, mínimo, VPCs y prefijos de recursos distintos por ambiente
RNF-08 Auditoría	Toda acción administrativa registrada	CloudTrail habilitado a nivel cuenta + logs entregados a bucket S3 con object lock
RNF-09 Costo controlado	Budget alert al 80% del presupuesto mensual	AWS Budgets + alarma a Slack del equipo
RNF-10 Reproducibilidad	Cualquier ambiente se levanta de cero con <code>terraform apply</code>	IaC al 100%, sin pasos manuales documentados fuera del repo

3.6 Configuración del cluster EKS

Aspecto	Configuración
Versión de Kubernetes	<code>1.30</code> o superior

Aspecto	Configuración
Tipo de nodos	Managed Node Groups con instancias <code>t3.medium</code> (dev) / <code>m5.large</code> (prod)
Estrategia	Mínimo 2 nodos por AZ en <code>prod</code> (3 AZs)
Networking	VPC CNI con prefix delegation; pods en subredes privadas
Ingress	AWS Load Balancer Controller (ALB Ingress) terminando en API Gateway VPC Link
Observabilidad	Contenedor sidecar / DaemonSet de CloudWatch agent + Fluent Bit
Identidad de pods	IRSA habilitado; cada workload tiene su <code>ServiceAccount</code> con role IAM dedicado
Autoescalado	Cluster Autoscaler + HPA por workload
Política de actualización	Rolling update con <code>maxSurge=25%</code> , <code>maxUnavailable=0</code>

3.7 Impacto sobre el plan de pruebas

Estos requerimientos cloud generan **nuevas superficies a probar** que se cubren en las secciones siguientes:

- **Pruebas de infraestructura como código:** `terraform plan` y validadores estáticos (tflint, tfsec, checkov) se ejecutan en CI — quedan recogidos como parte de las pruebas unitarias y de seguridad (§4 y §8).
- **Pruebas de integración contra servicios reales o emulados:** EKS local (kind o minikube) y LocalStack para AWS local en CI (§5).
- **Pruebas de seguridad sobre la superficie cloud:** roles IAM, exposición de buckets S3, políticas WAF, configuración de Security Groups (§8).
- **Pruebas de carga sobre API Gateway + EKS:** validar HPA, throttling de API Gateway y profundidad de SQS bajo carga (§9).
- **Pruebas de estrés sobre el cluster:** punto de quiebre del autoescalado, comportamiento ante fallo de AZ, saturación de RDS (§10).

4. Pruebas unitarias

4.1 Objetivo

Validar de forma aislada las unidades de lógica de dominio del backend: validadores de payload, cálculo de prioridad (§4.2), reglas de SLA (§4.3), evaluación de RBAC (§4.7), generación de URLs prefirmadas (§4.5) y construcción de eventos de historial (§4.4).

4.2 Scope

- Sin red, sin base de datos, sin S3 ni SQS reales: todo se aísla con dobles de prueba.
- Las dependencias externas (cliente S3, repositorio de tickets, reloj/clock) se inyectan para permitir aislamiento.

4.3 Herramientas

- **Backend Node/TypeScript** (preferente para el equipo): **Vitest** o **Jest**.
- **Backend Python** (alternativa): **Pytest** con `pytest-mock` y `freezegun`.
- Cobertura: **c8 / nyc** (Node) o **coverage.py** (Python).
- Reporte: **JUnit XML** subido como artefacto del pipeline.

4.4 Criterio de cobertura objetivo

- **>= 80% líneas y >= 75% ramas** en módulos de dominio (`/src/domain/*`, `/src/usecases/*`).
- **100% en funciones críticas:** cálculo de prioridad, evaluación de SLA, evaluación de RBAC.

4.5 Casos representativos

ID	Descripción	Precondición	Pasos	Resultado esperado	Prioridad
TU-001	Cálculo de prioridad: Incidente Crítico > Solicitud Crítica	Función <code>calcularPrioridad</code> cargada (§4.2)	Invocar con <code>(Incidente, Crítica)</code> y con <code>(Solicitud, Crítica)</code>	El valor numérico del incidente es estrictamente mayor que el de la solicitud	Alta
TU-002	Cálculo de prioridad: Crítica > Alta > Media > Baja para mismo tipo	—	Invocar las 4 combinaciones con <code>Incidente</code>	Orden estricto descendente	Alta
TU-003	Cálculo de prioridad: tipo inválido lanza <code>ValidationError</code>	—	Invocar con <code>(Otro, Alta)</code>	Excepción <code>ValidationError</code> con código <code>INVALID_TYPE</code>	Media
TU-004	SLA: incidente crítico vence a los 15 minutos sin actividad (§4.3)	Reloj fijado a <code>T0</code> ; ticket creado en <code>T0</code>	Avanzar reloj a <code>T0 + 14m</code> ; evaluar	<code>isSlaVencido === false</code>	Alta
TU-005	SLA: incidente crítico vence en exactamente 15 minutos	Idem TU-004	Avanzar a <code>T0 + 15m + 1s</code> ; evaluar	<code>isSlaVencido === true</code> , próximo nivel <code>L2</code>	Alta
TU-006	Escalamiento: <code>L1 → L2 → L3</code> ; no avanza más allá de <code>L3</code>	Ticket en <code>L3</code> con SLA vencido	Invocar <code>siguienteNivel</code>	Devuelve <code>L3</code> y marca <code>escalamientoMaximoAlcanzado=true</code>	Alta
TU-007	RBAC: Reportante no puede asignar tickets (§4.7)	Token con rol <code>Reportante</code>	Evaluar permiso para <code>assign</code>	<code>denegado=true</code> , razón <code>ROLE_NOT_ALLOWED</code>	Crítica
TU-008	RBAC: Agente puede cambiar estado de cualquier ticket	Token con rol <code>Agente</code>	Evaluar permiso para <code>changeState</code>	<code>permitido=true</code>	Crítica
TU-009	RBAC: Reportante solo accede a sus propios tickets	Token con rol <code>Reportante</code> , ticket de otro usuario	Evaluar permiso para <code>view(ticketAjeno)</code>	<code>denegado=true</code> , razón <code>NOT_OWNER</code>	Crítica
TU-010	URL prefirrada S3 (§4.5): se genera con expiración configurada	Cliente S3 mockeado	Invocar <code>presign(key, 300s)</code>	URL contiene parámetros <code>X-Amz-Expires=300</code>	Alta
TU-011	URL prefirrada: rechaza expiración > 1 hora	—	Invocar <code>presign(key, 7200s)</code>	Excepción <code>InvalidExpirationError</code>	Media
TU-012	Generación de número de ticket <code>TKT-XXXX</code> es autoincremental	Secuencia inicia en 0	Generar 3 tickets seguidos	<code>TKT-0001</code> , <code>TKT-0002</code> , <code>TKT-0003</code>	Media
TU-013	Validación: título obligatorio y máximo 200 caracteres	—	Crear ticket sin título y con 201 caracteres	Ambos rechazados con error de validación	Alta
TU-014	Validación: severidad solo acepta {Crítica, Alta, Media, Baja}	—	Crear ticket con <code>severidad="Urgente"</code>	Rechazado con <code>INVALID_SEVERITY</code>	Alta

ID	Descripción	Precondición	Pasos	Resultado esperado	Prioridad
TU-015	Historial (§4.4): el evento queda inmutable — intento de update lanza error	Repo de eventos en modo "append-only"	Intentar <code>update(event)</code>	Excepción <code>ImmutableEventError</code>	Alta
TU-016	Resolución requiere causa raíz y solución aplicada (CU-03)	Ticket en <code>EnProgreso</code>	Cambiar a <code>Resuelto</code> sin esos campos	Excepción <code>MissingResolutionFieldsError</code>	Crítica
TU-017	Cálculo de tiempo de resolución para reporte (CU-07)	Ticket creado en T0, resuelto en T0+3h	Invocar <code>tiempoResolucion(ticket)</code>	10800 segundos	Media
TU-018	Reloj UTC: ningún timestamp usa zona local	—	Generar evento; inspeccionar timestamp	El timestamp termina en Z (UTC)	Media

4.6 Criterios de aprobación

- 100% de los casos pasan en CI antes de mergear a `main`.
- Cobertura $\geq$ umbrales declarados en §4.4 (falla del pipeline si no se cumple).
- Tiempo total de la suite unitaria < 60 segundos.

5. Pruebas de backend (integración de API)

5.1 Objetivo

Validar el comportamiento extremo a extremo de cada endpoint REST de §5 contra **dependencias reales o equivalentes**: PostgreSQL real (testcontainers o RDS efímero), SQS real (LocalStack o cola dedicada) y S3 real (LocalStack o bucket dedicado).

5.2 Scope

- Endpoints en §5 de E1: `POST /tickets`, `PATCH /tickets/{id}/state`, `POST /tickets/{id}/assign`, `GET /tickets?filters=...`, además de los derivados: `POST /tickets/{id}/comments`, `POST /tickets/{id}/attachments`, `GET /tickets/{id}/history`.
- Casos felices, errores controlados (4xx), errores del servidor (5xx), idempotencia y concurrencia (optimistic locking — pregunta abierta §6 de E1, se prueba el comportamiento esperado).

5.3 Herramientas

- **Supertest** + **Vitest/Jest** (Node) o **pytest** + **httpx** (Python).
- **Testcontainers** para Postgres, LocalStack para S3/SQS.
- **Pact** o **Schemathesis** para contract testing contra la especificación OpenAPI cuando esté disponible (D2/D3).
- Datos sintéticos: **Faker** + factorías propias por entidad.

5.4 Casos representativos

5.4.1 `POST /tickets` (CU-01, CU-08, §4.1, §4.2)

ID	Descripción	Precondición	Pasos	Resultado esperado	Prioridad
TI-001	Crear incidente crítico devuelve 201 con <code>TKT-XXXX</code>	Token de <code>Reportante</code> válido	POST con <code>tipo=Incidente</code> , <code>severidad=Crítica</code> , título, descripción	201; body con <code>id TKT-XXXX</code> , <code>estado=Abierto</code> , <code>prioridad</code> calculada, <code>createdAt</code> UTC	Crítica
TI-002	Crear solicitud va a cola separada	Token válido	POST con <code>tipo=Solicitud</code> , <code>severidad=Media</code>	201; <code>tipo=Solicitud</code> ; GET de la cola separada incluye el ticket	Alta

ID	Descripción	Precondición	Pasos	Resultado esperado	Prioridad
TI-003	Crear ticket sin título devuelve 400	Token válido	POST sin <code>título</code>	400 con detalle del campo faltante	Alta
TI-004	Crear ticket sin token devuelve 401	—	POST sin header <code>Authorization</code>	401 <code>UNAUTHENTICATED</code>	Crítica
TI-005	Crear ticket con token expirado devuelve 401	Token <code>exp</code> en el pasado	POST	401 <code>TOKEN_EXPIRED</code>	Crítica
TI-006	Crear ticket persiste evento <code>TICKET_CREATED</code> en historial	—	POST; luego GET <code>/tickets/{id}/history</code>	El historial incluye un único evento <code>TICKET_CREATED</code> con autor y timestamp UTC	Alta
TI-007	Idempotencia: dos POST con mismo <code>Idempotency-Key</code> crean un único ticket	—	POST con header <code>Idempotency-Key: abc</code> ; repetir	Segundo POST devuelve 200 con el mismo <code>TKT-XXXX</code>	Alta
TI-008	Payload con campos extra ignorados (no rompe)	—	POST con <code>tipo</code> , <code>severidad</code> , <code>extraField</code>	201; el campo extra no se persiste	Baja
TI-009	Adjuntos: el ticket se vincula a los keys S3 informados (§4.5)	Archivo previamente subido vía URL prefirmada	POST con <code>attachmentKeys=[...]</code>	201; GET del ticket devuelve los adjuntos	Alta

5.4.2 PATCH `/tickets/{id}/state` (CU-03, §4.4)

ID	Descripción	Precondición	Pasos	Resultado esperado	Prioridad
TI-020	Agente cambia <code>Abierto</code> → <code>EnProgreso</code> con comentario	Token de Agente; ticket existe en <code>Abierto</code>	PATCH con <code>estado=EnProgreso</code> , <code>comentario</code>	200; nuevo evento en historial con autor y timestamp	Crítica
TI-021	Resolución requiere causa raíz y solución (CU-03)	Ticket en <code>EnProgreso</code>	PATCH <code>estado=Resuelto</code> sin campos	422 <code>MISSING_RESOLUTION_FIELDS</code>	Crítica
TI-022	Resolución completa cierra el ticket y notifica	Ticket en <code>EnProgreso</code>	PATCH <code>estado=Resuelto</code> con causa raíz y solución	200; mensaje publicado en SQS para notificar al reportante	Alta
TI-023	Transición ilegal <code>Abierto</code> → <code>Resuelto</code> directamente	Ticket en <code>Abierto</code>	PATCH <code>estado=Resuelto</code>	409 <code>ILLEGAL_STATE_TRANSITION</code>	Alta
TI-024	Reportante NO puede cambiar estado (RBAC §4.7)	Token de Reportante	PATCH	403 <code>ROLE_NOT_ALLOWED</code>	Crítica
TI-025	Concurrencia (pregunta abierta §6): dos PATCH simultáneos con <code>version</code> (optimistic lock)	Ticket con <code>version=1</code> ; cliente A y B leen	A hace PATCH primero (ok); B hace PATCH con <code>version=1</code>	A devuelve 200 con <code>version=2</code> ; B devuelve 409 <code>STALE_VERSION</code>	Alta
TI-026	Cambio de estado dispara evento en historial inmutable	—	PATCH <code>estado=EnProgreso</code>	Historial gana evento <code>STATE_CHANGED</code> ; intento de update directo en DB es bloqueado por trigger/RLS	Alta

5.4.3 POST `/tickets/{id}/assign` (CU-02)

ID	Descripción	Precondición	Pasos	Resultado esperado	Prioridad
TI-040	Agente se autoasigna un ticket de la cola	Token de Agente; ticket <code>Abierto</code> sin asignar	POST <code>assigneeId=self</code>	200; ticket pasa a <code>EnProgreso</code> ; notificación encolada al reportante	Crítica
TI-041	Asignar a agente inexistente devuelve 404	—	POST con <code>assigneeId</code> no existente	404 <code>ASSIGNEE_NOT_FOUND</code>	Alta

ID	Descripción	Precondición	Pasos	Resultado esperado	Prioridad
TI-042	Asignar a Reportante (rol incorrecto) devuelve 422	Usuario con rol Reportante	POST con <code>assigneeId=ese</code>	422 <code>INVALID_ASSIGNEE_ROLE</code>	Alta
TI-043	Reasignación: A→B notifica a ambos	Ticket asignado a A	POST <code>assigneeId=B</code>	200; SQS recibe dos eventos de notificación	Media

5.4.4 GET /tickets?filters=... (CU-06)

ID	Descripción	Precondición	Pasos	Resultado esperado	Prioridad
TI-060	Filtro por estado retorna solo coincidencias	Dataset con 50 tickets en varios estados	GET <code>?estado=Abierto</code>	200; todos los items tienen <code>estado=Abierto</code>	Alta
TI-061	Filtros combinados: estado + prioridad + agente	Dataset variado	GET <code>?estado=EnProgreso&prioridad=Alta&assignee=u1</code>	200; todos los items cumplen las 3 condiciones	Alta
TI-062	Paginación con <code>?limit=20&cursor=...</code>	100 tickets	GET con cursor en página 2	200; 20 items; <code>nextCursor</code> presente	Media
TI-063	Latencia P95 con filtros < 2 s (criterio CU-06)	10k tickets sembrados	Repetir 100 GETs concurrentes	P95 < 2000 ms	Crítica
TI-064	Reportante solo ve sus tickets (RBAC §4.7)	Token de Reportante con 3 tickets propios; dataset con 100 ajenos	GET sin filtros	200; solo los 3 propios	Crítica

5.4.5 Adjuntos y URLs prefiradas (§4.5)

ID	Descripción	Precondición	Pasos	Resultado esperado	Prioridad
TI-080	Solicitar URL prefirada de subida	Token válido	POST <code>/attachments/presign</code> con <code>fileName</code> , <code>contentType</code>	200; URL S3 con expiración 5 min	Alta
TI-081	La URL prefirada deja de funcionar al expirar	URL emitida con 5 s de TTL	Esperar 10 s; intentar PUT	403 del S3 (firma expirada)	Crítica
TI-082	Limitar <code>Content-Type</code> a <code>{image/png, image/jpeg, application/pdf, text/plain}</code>	—	Solicitar presign con <code>application/x-msdownload</code>	422 <code>UNSUPPORTED_CONTENT_TYPE</code>	Alta

5.4.6 Asíncrono — SQS y workers (§4.6, §4.3, CU-04)

ID	Descripción	Precondición	Pasos	Resultado esperado	Prioridad
TI-100	El worker de notificaciones consume el mensaje y marca como procesado	Mensaje encolado por creación de ticket	Arrancar worker; observar cola	Mensaje consumido; entrada en log estructurado; cola vacía	Alta
TI-101	Reintento con backoff ante fallo del proveedor de email	Proveedor mock devuelve 500 dos veces y luego 200	Encolar 1 mensaje	El mensaje se reintenta 2 veces y se entrega en el tercer intento	Alta
TI-102	Mensaje no procesable termina en DLQ luego de N reintentos	Proveedor devuelve siempre 500	Encolar 1 mensaje	Tras 5 intentos el mensaje aparece en la DLQ	Alta
TI-103	Job de escalamiento eleva ticket crítico sin actividad de L1 a L2	Ticket crítico creado hace 16 min, sin cambios	Disparar job	Ticket queda en L2 ; evento <code>ESCALATED</code> en historial; notificación encolada a admin	Crítica

ID	Descripción	Precondición	Pasos	Resultado esperado	Prioridad
TI-104	Job de escalamiento NO escala tickets con actividad reciente	Ticket crítico con comentario hace 5 min	Disparar job	Ticket sigue en L1 ; sin nuevos eventos	Alta

5.5 Criterios de aprobación

- 100% de los casos P0/P1 pasan en staging antes de UAT.
- Latencia P95 de cada endpoint en condiciones nominales < 500 ms (excepto reportes CU-07, < 3 s).
- 0 defectos abiertos con severidad Crítica/Alta al cierre de la fase.

6. Pruebas E2E

6.1 Objetivo

Validar flujos completos de usuario que atraviesen el sistema de punta a punta: autenticación, creación de ticket, asignación, resolución y notificación.

6.2 Scope

Los 8 CUs priorizados (CU-01 a CU-08). Dado que el frontend productivo está fuera de scope (§6 de E1), los flujos E2E **se ejecutan contra el API** simulando la interacción del FE, complementados con recorridos manuales sobre los mockups para validar el flujo visual.

6.3 Herramientas

- **Playwright** (preferente) para los escenarios que en una entrega futura tendrán FE; corre hoy contra el API con request context.
- **Cypress** como alternativa si el equipo opta por él.
- **Newman** (Postman CLI) para flujos puramente HTTP en CI.
- **Mailhog** o **Mailpit** como receptor de email durante pruebas; **canal Slack de QA** dedicado.

6.4 Casos representativos

ID	Descripción	CU	Pasos resumidos	Resultado esperado	Prioridad
TE2E-001	Reportante crea incidente crítico y recibe confirmación	CU-01	Login → POST ticket → consulta GET	Ticket TKT-XXXX ; estado Abierto ; email/Slack de confirmación recibido	Crítica
TE2E-002	Agente toma ticket de la cola y reportante es notificado	CU-02	Login agente → GET cola → POST assign → consultar notificaciones	Ticket EnProgreso ; reportante recibe notificación con nombre del agente	Crítica
TE2E-003	Agente resuelve ticket con causa raíz y solución	CU-03	PATCH estado=Resuelto con campos requeridos	Ticket Resuelto ; evento en historial; reportante notificado	Crítica
TE2E-004	Escalamiento automático por SLA vencido	CU-04	Crear crítico → esperar 16 min (o avanzar reloj) → disparar job	Ticket pasa a L2 ; admin recibe notificación; historial registra ESCALATED	Crítica
TE2E-005	Consulta de historial completo del ticket	CU-05	Recorrido completo de un ticket → GET /history	Lista cronológica de eventos con autor y timestamp; adjuntos accesibles vía URL prefirmada	Alta
TE2E-006	Filtrado de la cola por estado + prioridad + agente	CU-06	GET con combinación de filtros	Resultados consistentes con el filtro; latencia < 2 s	Alta
TE2E-007	Reporte de resolución por período descargable	CU-07	Login admin → GET /reports?from=...&to=...&format=csv	CSV descargable; agregados correctos validados contra dataset sembrado	Media

ID	Descripción	CU	Pasos resumidos	Resultado esperado	Prioridad
TE2E-008	Solicitud de servicio en cola separada del incidente	CU-08	POST tipo=Solicitud → GET cola incidentes + GET cola solicitudes	La solicitud aparece solo en la cola de solicitudes	Media
TE2E-009	Recorrido visual sobre mockups: crear ticket → cola → detalle	CU-01, CU-02, CU-03	Guiar al validador por los mockups 3, 2, 5	El flujo se entiende sin instrucciones adicionales	Media
TE2E-010	Recorrido visual: escalamiento y reporte	CU-04, CU-07	Mockups 7 y 8	El validador identifica nivel L1/L2/L3 y métricas del reporte	Media

6.5 Criterios de aprobación

- 100% de los E2E P0 verdes en `staging` antes de la demo.
- Tiempo total de la suite E2E < 15 minutos en CI.

7. Pruebas de aceptación (UAT)

7.1 Objetivo

Demostrar al cliente/profesor que cada CU cumple su **criterio de éxito declarado en §3 de E1**, en formato Gherkin verificable.

7.2 Scope

Una historia de aceptación por cada CU (mínimo); escenarios adicionales por variantes relevantes.

7.3 Herramientas

- **Cucumber** / **SpecFlow** / **Behave** para automatización (opcional).
- **Documentación viva** (ej. living documentation) generada desde los `.feature` y publicada en `docs/uat/`.
- Validación final con el profesor durante la demo.

7.4 Escenarios Gherkin

```
# TUAT-001 · CU-01 – Abrir un ticket de incidente
Feature: Abrir un ticket de incidente
  Como reportante
  Quiero registrar un incidente con título, descripción, severidad y adjuntos
  Para que el equipo de operaciones tenga el contexto necesario

  Scenario: Crear incidente crítico con adjuntos
    Given estoy autenticado como Reportante
    And he subido un adjunto "captura.png" mediante una URL prefirmada
    When envío un POST /tickets con tipo "Incidente", severidad "Crítica",
      título "API caída en prod" y los keys del adjunto
    Then recibo respuesta 201
    And el cuerpo contiene un id con formato "TKT-XXXX"
    And el estado del ticket es "Abierto"
    And el timestamp de creación está en UTC
    And el adjunto queda accesible mediante URL prefirmada con expiración
    And recibo notificación de confirmación
```

```
# TUAT-002 · CU-02 – Clasificar y asignar
Feature: Clasificar y asignar un ticket
  Scenario: Agente asigna ticket crítico desde la cola priorizada
    Given existe un ticket "TKT-0042" Incidente Crítico en estado "Abierto"
    And estoy autenticado como Agente
    When solicito GET /tickets ordenado por prioridad
    Then "TKT-0042" aparece encabezando la lista
    When envío POST /tickets/TKT-0042/assign con mi userId
    Then el ticket pasa a estado "EnProgreso"
    And el reportante recibe notificación indicando quién lo atiende
```

```
# TUAT-003 · CU-03 – Resolución con causa raíz y solución
Feature: Registrar resolución
  Scenario: Resolver ticket con campos obligatorios
    Given existe un ticket "TKT-0042" en estado "EnProgreso"
    And estoy autenticado como Agente
    When envío PATCH /tickets/TKT-0042/state con estado "Resuelto",
      causa raíz "Memoria saturada en pod" y solución "Reinicio + ajuste de límites"
    Then recibo respuesta 200
    And el historial incluye un evento "RESOLVED" con mis campos
    And el reportante recibe notificación de resolución
```

```
# TUAT-004 · CU-04 – Escalamiento automático por SLA
Feature: Escalamiento automático
  Scenario: SLA vencido eleva un ticket de L1 a L2
    Given existe un ticket "TKT-0043" Incidente Crítico creado hace 16 minutos
    And el SLA configurado para Incidente Crítico es 15 minutos
    And no ha habido actividad sobre el ticket
    When el job de escalamiento se ejecuta
    Then el ticket queda en nivel "L2"
    And el agente asignado y el administrador reciben notificación
    And el historial registra evento "ESCALATED" con timestamp
```

```
# TUAT-005 · CU-05 – Historial y trazabilidad
Feature: Historial inmutable
  Scenario: Consultar timeline ordenado cronológicamente
    Given existe un ticket "TKT-0044" con 5 eventos
    When solicito GET /tickets/TKT-0044/history
    Then recibo los 5 eventos ordenados por timestamp ascendente
    And cada evento contiene autor, timestamp UTC y descripción
    And los adjuntos referenciados son accesibles vía URL prefirmada
```

```
# TUAT-006 · CU-06 – Filtrado en menos de 2 segundos
Feature: Filtrado de la cola
  Scenario: Filtros combinados con latencia menor a 2 segundos
    Given existen al menos 10000 tickets en el sistema
    When solicito GET /tickets?estado=Abierto&prioridad=Alta&assignee=u1
    Then recibo los resultados que cumplen los tres filtros
    And la respuesta tarda menos de 2 segundos
```

```
# TUAT-007 · CU-07 – Reporte por período
Feature: Reporte agregado
  Scenario: Generar reporte de un mes y descargar CSV
    Given soy Administrador
    When solicito GET /reports?from=2026-05-01&to=2026-05-31&format=csv
    Then recibo un CSV descargable
    And el CSV contiene cantidad de resueltos, tiempo promedio de resolución,
      distribución por categoría y por agente
```

```
# TUAT-008 · CU-08 – Solicitud de servicio en cola separada
Feature: Solicitudes de servicio
  Scenario: La solicitud no aparece en la cola de incidentes
    Given soy Reportante
    When envío POST /tickets con tipo "Solicitud", severidad "Media"
    Then el ticket queda con tipo "Solicitud"
    And aparece en la cola de solicitudes
    And NO aparece en la cola de incidentes
```

7.5 Criterios de aprobación

- Aprobación firmada por el profesor/cliente para cada uno de los 8 escenarios principales.
- Defectos derivados clasificados y priorizados antes del cierre.

8. Pruebas de seguridad

8.1 Objetivo

Verificar que el sistema cumple las garantías de seguridad declaradas (§4.7, §5 Seguridad de E1) y que no es vulnerable a las amenazas más relevantes del OWASP Top 10 para una API REST con almacenamiento de archivos.

8.2 Scope

- RBAC por rol y por endpoint (§4.7).
- Validación de JWT: firma, expiración, claims requeridos, rotación de claves.
- URLs prefiradas: expiración real, alcance restringido (un solo objeto, un solo verbo), prevención de re-uso.
- Validación de inputs y prevención de inyecciones (SQL, NoSQL, comandos, headers).
- IDOR sobre tickets ajenos.
- SSRF a través de adjuntos (URLs remotas, redirecciones).
- Headers de seguridad (HSTS, CSP, X-Content-Type-Options, X-Frame-Options, Referrer-Policy).
- Manejo seguro de secretos: que no aparezcan en logs ni en respuestas de error.
- Imagen del contenedor: vulnerabilidades conocidas.

8.3 Herramientas

- **OWASP ZAP** (active + passive scan) contra `staging`.
- **Burp Suite Community** para exploración manual y manipulación de requests.
- **sqlmap** para pruebas dirigidas de inyección SQL.
- **Trivy** para escaneo de la imagen del contenedor de ECS Fargate.
- **gitleaks** / **truffleHog** para escaneo de secretos en el repo.
- **JWT debugger** + scripts propios para forjado de tokens.
- **Lighthouse** y **securityheaders.com** para headers HTTP.

8.4 Casos representativos

ID	Descripción	Categoría	Pasos	Resultado esperado	Prioridad
TSEC-001	RBAC: Reportante no puede listar todos los tickets	Broken Access Control	GET /tickets con token de Reportante	200 que solo incluye sus propios tickets, nunca de otros	Crítica
TSEC-002	IDOR: Reportante intenta leer ticket ajeno por ID	Broken Access Control	GET /tickets/TKT-AJENO	403 o 404 (consistente) — nunca 200 con contenido	Crítica
TSEC-003	Reportante intenta asignar un ticket	Broken Access Control	POST /tickets/{id}/assign con token de Reportante	403 <code>ROLE_NOT_ALLOWED</code>	Crítica

ID	Descripción	Categoría	Pasos	Resultado esperado	Prioridad
TSEC-004	Reportante intenta llamar <code>/admin/sla-rules</code>	Broken Access Control	GET con token de Reportante	403; mensaje sin filtración de existencia del recurso	Crítica
TSEC-005	JWT: firma alterada → rechazado	Broken Authentication	Modificar payload del JWT sin recalcular firma	401 <code>INVALID_TOKEN</code>	Crítica
TSEC-006	JWT: algoritmo <code>none</code> → rechazado	Broken Authentication	Token con header <code>alg=none</code>	401; el servidor jamás acepta <code>none</code>	Crítica
TSEC-007	JWT: expirado → rechazado	Broken Authentication	Token con <code>exp</code> en el pasado	401 <code>TOKEN_EXPIRED</code>	Crítica
TSEC-008	JWT: escalada de rol — cambiar claim <code>role</code> a <code>Administrador</code>	Broken Authentication	Forjar token con rol elevado sin firma válida	401	Crítica
TSEC-009	URL prefirmada expira realmente	Cryptographic Failures	Pedir presign con TTL 5 s; esperar 10 s; PUT	403 del S3	Alta
TSEC-010	URL prefirmada solo permite el verbo solicitado	Cryptographic Failures	Pedir URL para PUT; intentar GET sobre la misma URL	403 / 400 del S3	Alta
TSEC-011	SQL injection en filtros de <code>GET /tickets</code>	Injection	GET <code>?titulo=' OR 1=1 --</code>	200 sin retornar todos los registros; query parametrizada en logs	Crítica
TSEC-012	NoSQL / template injection en campos de texto	Injection	POST con <code>titulo="{{7*7}}"</code> y con <code>\$ne</code>	El valor se almacena literal; ninguna evaluación	Alta
TSEC-013	XSS almacenado en comentarios (entrada peligrosa preservada como texto)	Injection / XSS	POST comentario con <code><script>alert(1)</script></code>	El contenido se devuelve escapado; jamás ejecutado	Alta
TSEC-014	SSRF: adjunto con URL remota que apunta a metadata <code>169.254.169.254</code>	SSRF	POST adjunto con URL remota a metadata EC2	Rechazado por allowlist / bloqueo de IPs internas	Crítica
TSEC-015	Validación de tipo MIME real del archivo subido	Insecure Design	Subir <code>malware.exe</code> renombrado a <code>.png</code>	Rechazado al validar firma del archivo, no solo extensión	Alta
TSEC-016	Headers de seguridad presentes en respuestas	Security Misconfiguration	GET cualquier endpoint	Headers <code>Strict-Transport-Security</code> , <code>X-Content-Type-Options: nosniff</code> , <code>Referrer-Policy</code> , <code>Content-Security-Policy</code>	Alta
TSEC-017	TLS 1.2+ obligatorio; TLS 1.0/1.1 rechazado	Cryptographic Failures	Cliente con TLS 1.0	Conexión rechazada	Alta
TSEC-018	Logs no contienen JWT, password ni cuerpo de tokens	Logging / Sensitive Data	Provocar error 500; revisar logs	Logs incluyen <code>requestId</code> pero ningún token completo ni dato sensible	Crítica
TSEC-019	Mensajes de error no filtran stack trace en prod	Security Misconfiguration	Provocar error con payload mal formado	Response 400/500 con mensaje genérico; sin stack	Alta
TSEC-020	Rate limiting en <code>POST /tickets</code>	DoS / Abuso	1000 POST en 10 s desde una IP	429 al exceder el umbral	Media
TSEC-021	Bucket S3 sin acceso público	Cloud config	Listar el bucket sin credenciales	Acceso denegado; no permite <code>s3:ListBucket</code> anónimo	Crítica
TSEC-022	Imagen de contenedor sin vulnerabilidades High/Critical	Vulnerable Dependencies	<code>trivy image <image></code>	0 vulnerabilidades High/Critical conocidas	Alta
TSEC-023	Secretos en repo	Sensitive Data Exposure	<code>gitleaks detect</code>	0 hallazgos	Crítica

8.5 Criterios de aprobación

- **0 vulnerabilidades Críticas y 0 Altas** abiertas al cierre.
- Reporte ZAP firmado y archivado.
- Reporte Trivy con conteo de hallazgos por severidad incluido en el deploy.

9. Pruebas de carga

9.1 Objetivo

Verificar que los endpoints críticos sostienen la carga esperada en horas pico, manteniendo latencia P95 dentro de SLA y sin errores 5xx.

9.2 Scope

Endpoints más críticos:

- `POST /tickets` (CU-01, CU-08).
- `GET /tickets?filters=...` con dataset realista (CU-06; SLA < 2 s).
- `PATCH /tickets/{id}/state` (CU-03).
- `GET /tickets/{id}/history` (CU-05).

9.3 Herramientas

- **k6** (preferente) por su pipeline-friendliness y reporte en JSON.
- **Locust** o **JMeter** como alternativas si el equipo ya las conoce.
- Métricas exportadas a **CloudWatch** (latencia P95, errores 5xx) y a un dashboard en Grafana (opcional).

9.4 Baseline esperado (asunciones del equipo)

Asumiendo una organización de 50–200 ingenieros (definido en E1 §1):

Métrica	Valor esperado
Tickets creados / día	150–400
Usuarios concurrentes en horario laboral	30–60
RPS pico estimado en <code>POST /tickets</code>	5 req/s
RPS pico estimado en <code>GET /tickets</code> (refresco de cola por polling)	20 req/s

9.5 Casos representativos

ID	Descripción	Carga	Duración	Criterio de aprobación	Prioridad
TC-001	<code>POST /tickets</code> sostenido	5 VU constantes con ramp-up 1 min	15 min	P95 < 500 ms; error rate < 0.1%	Crítica
TC-002	<code>POST /tickets</code> pico	Ramp 1 → 25 VU en 2 min	10 min	P95 < 1000 ms; error rate < 1%	Alta
TC-003	<code>GET /tickets</code> con filtros	20 VU constantes	15 min	P95 < 2000 ms (SLA CU-06); error rate < 0.1%	Crítica
TC-004	<code>GET /tickets</code> filtros + 10k tickets sembrados	20 VU	15 min	P95 < 2000 ms	Crítica
TC-005	<code>PATCH /tickets/{id}/state</code>	5 VU	10 min	P95 < 500 ms; 0 errores de optimistic lock fuera de los esperados	Alta
TC-006	Carga mixta proporcional al uso real	70% GET, 20% POST, 10% PATCH; 30 VU	30 min	P95 global < 1000 ms; error rate < 0.5%	Alta

ID	Descripción	Carga	Duración	Criterio de aprobación	Prioridad
TC-007	Latencia E2E del flujo de notificación (SQS → worker → entrega)	50 mensajes/min	15 min	Tiempo P95 de entrega < 30 s	Alta
TC-008	Job de escalamiento bajo carga	1000 tickets críticos elegibles	1 ejecución	El job completa en < 60 s; ningún ticket queda sin procesar	Alta

9.6 Métricas y criterios

- Reporte por endpoint: P50, P95, P99, error rate, throughput.
- Aprobación** si: P95 < SLA por endpoint, error rate < 1% en carga pico, sin saturación de RDS (CPU < 80% , Connections < 80% del límite).
- Fallo** si: P95 supera SLA en cualquier endpoint crítico o aparece error rate sostenida > 2%.

10. Pruebas de estrés

10.1 Objetivo

Encontrar el **punto de quiebre** del sistema y verificar que la degradación sea controlada y reversible (graceful degradation).

10.2 Scope

- Saturación de SQS (mensajes que se acumulan más rápido de lo que el worker consume).
- Agotamiento del connection pool de RDS.
- Presión sobre el worker de escalamiento con miles de tickets elegibles.
- Comportamiento ante 10x el tráfico esperado.

10.3 Herramientas

Mismas que carga (k6/Locust/JMeter), más:

- AWS Fault Injection Simulator** (opcional) para introducir latencia controlada en RDS o S3.
- CloudWatch alarms** para validar que las alertas declaradas en §5 Observabilidad disparan a tiempo.

10.4 Casos representativos

ID	Descripción	Carga / fault	Criterio de degradación aceptable	Prioridad
TS-001	Sobrecarga 10x en POST /tickets	50 VU sostenidos	El sistema responde 429 a las peticiones excedentes; no entrega 5xx > 5%; recupera P95 normal < 2 min tras parar la carga	Alta
TS-002	Sobrecarga 10x en GET /tickets	200 VU sostenidos	P95 puede degradar a < 5 s; error rate < 5%; sin caída total	Alta
TS-003	SQS: encolar 10000 mensajes en 1 min con un solo worker	Workers = 1	La cola se vacía con escalado horizontal manual a 5 workers; ningún mensaje se pierde	Alta
TS-004	RDS: forzar agotamiento del pool de conexiones	Reducir pool a 5 mientras corre TC-003	La API responde 503 limpio; recupera al liberar el pool	Alta
TS-005	Job de escalamiento con 5000 tickets críticos vencidos	Sembrado masivo	El job procesa todo sin agotar memoria del worker; tiempo < 5 min	Alta
TS-006	Caída simulada de S3 al subir adjunto	Bloquear endpoint S3	La API responde 503 al usuario con mensaje claro; el ticket NO se crea inconsistente	Crítica
TS-007	Caída del proveedor de email	Mock devuelve 500 en todas las llamadas	Los mensajes se reintentan y caen a DLQ; el resto del sistema sigue operativo	Alta
TS-008	Carga sostenida durante 1 hora (soak test)	30 VU mixto	Sin memory leak observable; latencia estable; sin crecimiento sostenido de errores	Media

10.5 Criterios de aprobación

- El sistema **no entra en cascada de fallos** bajo ningún escenario.
- Tras retirar la sobrecarga, el sistema recupera el estado nominal en < 5 min.
- Las alarmas declaradas en §5 Observabilidad de E1 disparan correctamente en cada escenario.

11. Pruebas de experiencia de usuario (UX)

11.1 Objetivo

Evaluar la usabilidad de los 8 mockups en `docs/mockups/` aplicando heurísticas de Nielsen y criterios básicos de accesibilidad WCAG 2.1 AA. Validar que los flujos críticos (crear ticket, identificar SLA vencido, comprender escalamiento L1/L2/L3, leer un reporte) se entienden sin instrucciones adicionales.

11.2 Scope

#	Mockup	CU(s)	Foco de evaluación
1	Login	RBAC §4.7	Claridad de errores; campos accesibles
2	Cola de tickets con filtros	CU-02, CU-06	Filtros visibles, badge "SLA vencido", densidad de información
3	Crear ticket — Incidente	CU-01	Etiquetas claras; campos obligatorios marcados; subida de adjuntos comprensible
4	Crear ticket — Solicitud	CU-08	Diferenciación visual frente al incidente
5	Detalle + registrar resolución	CU-03	Causa raíz y solución como campos obligatorios visibles
6	Historial / timeline	CU-05	Ordenamiento cronológico evidente; descarga de adjuntos clara
7	Tickets escalados por SLA	CU-04	Distinción clara entre L1/L2/L3; razón del escalamiento
8	Reportes por período	CU-07	KPI cards legibles; descarga CSV visible

11.3 Herramientas

- **Evaluación heurística** (10 heurísticas de Nielsen) por al menos 2 evaluadores.
- **axe-core** o **Lighthouse Accessibility** sobre prototipos navegables (cuando los haya).
- **Contrast Checker** (WebAIM) para validar contraste WCAG AA (4.5:1 texto normal, 3:1 texto grande).
- **Navegación por teclado** manual (Tab, Shift+Tab, Enter, Esc).
- **Lectores de pantalla** (NVDA en Windows, VoiceOver en macOS) sobre prototipos navegables.

11.4 Casos representativos

ID	Descripción	Mockup	Criterio	Prioridad
TUX-001	Heurística "Visibilidad del estado del sistema" en cola	#2	La cola muestra claramente cuántos tickets están en cada estado	Alta
TUX-002	Heurística "Coincidencia con el mundo real": términos del dominio	Todos	Etiquetas en español y consistentes con el glosario (Reportante, Agente, Incidente, Solicitud)	Alta
TUX-003	Heurística "Prevención de errores" en creación de ticket	#3, #4	Campos obligatorios marcados; validación previa al submit	Alta
TUX-004	Heurística "Reconocimiento antes que recuerdo" en filtros	#2	Filtros aplicados visibles como chips que se pueden quitar	Media
TUX-005	Heurística "Flexibilidad y eficiencia" — atajos de teclado en creación	#3	Existe atajo para submit (Ctrl+Enter) o se indica	Baja
TUX-006	Heurística "Ayuda a reconocerse, diagnosticar y recuperarse de errores"	#3	Errores muestran qué campo y cómo corregirlo	Alta

ID	Descripción	Mockup	Criterio	Prioridad
TUX-007	Badge "SLA vencido" visible y comprensible	#2, #7	El badge es distinguible por forma y color, no solo color	Crítica
TUX-008	Distinción L1 vs L2 vs L3 en vista de escalamiento	#7	El nivel se identifica sin leer ayuda contextual	Alta
TUX-009	Causa raíz y solución marcados como obligatorios	#5	Ambos campos llevan asterisco/marca visible	Crítica
TUX-010	Contraste de texto cumple WCAG AA	Todos	Texto normal >= 4.5:1; texto grande >= 3:1	Alta
TUX-011	Navegación completa por teclado	Todos	Cada control alcanzable con Tab; foco visible	Alta
TUX-012	Labels asociados a inputs (accesibilidad)	#1, #3, #4, #5	Cada input tiene label o aria-label	Alta
TUX-013	Idioma declarado de la página	Todos	<html lang="es">	Media
TUX-014	Mensajes de notificación no dependen solo del color	#2, #7	Iconografía + texto + color	Media
TUX-015	Comprensión del reporte por período	#8	Un usuario sin contexto identifica las 3 métricas principales en < 30 s	Media

11.5 Criterios de aprobación

- **0 violaciones bloqueantes** de heurísticas (severidad 4 — catastrófica — en escala de Nielsen).
- **0 incumplimientos críticos** de WCAG AA en los mockups validados.
- Validación por al menos 2 evaluadores independientes.

12. Matriz de trazabilidad

Cada celda con ☒ indica que el tipo de prueba cubre el CU o la funcionalidad. Las celdas vacías significan que el tipo no aplica o se cubre indirectamente desde otro nivel.

12.1 CUs vs tipos de prueba

CU	Unitarias	Integración	E2E	UAT	Seguridad	Carga	Estrés	UX
CU-01 — Abrir ticket	✓	✓	✓	✓	✓	✓	✓	✓
CU-02 — Clasificar y asignar	✓	✓	✓	✓	✓	✓		✓
CU-03 — Actualizar estado y resolver	✓	✓	✓	✓	✓	✓		✓
CU-04 — Escalamiento por SLA	✓	✓	✓	✓			✓	✓
CU-05 — Historial y trazabilidad	✓	✓	✓	✓	✓	✓		✓
CU-06 — Filtrar y buscar	✓	✓	✓	✓	✓	✓	✓	✓
CU-07 — Reporte por período		✓	✓	✓	✓			✓
CU-08 — Solicitud de servicio	✓	✓	✓	✓	✓			✓

12.2 Funcionalidades §4.x vs tipos de prueba

§4.x	Unitarias	Integración	E2E	UAT	Seguridad	Carga	Estrés	UX
§4.1 Creación con severidad/tipo	✓	✓	✓	✓	✓	✓		✓

§4.x	Unitarias	Integración	E2E	UAT	Seguridad	Carga	Estrés	UX
§4.2 Priorización automática	✓	✓	✓	✓		✓		✓
§4.3 Escalamiento por SLA	✓	✓	✓	✓			✓	✓
§4.4 Historial inmutable	✓	✓	✓	✓	✓			✓
§4.5 Adjuntos en S3 / presign	✓	✓	✓	✓	✓		✓	✓
§4.6 Notificaciones asíncronas	✓	✓	✓	✓	✓	✓	✓	
§4.7 RBAC	✓	✓	✓	✓	✓			✓

12.3 Componentes AWS (§5 E1) vs tipos de prueba

Componente	Unitarias	Integración	Carga	Estrés	Seguridad
API en ECS Fargate	✓	✓	✓	✓	✓
RDS PostgreSQL		✓	✓	✓	✓
S3 (adjuntos)	✓	✓		✓	✓
SQS + worker		✓	✓	✓	
Job de escalamiento		✓		✓	
VPC + Security Groups					✓
JWT + RBAC	✓	✓			✓

13. Criterios de aceptación (entrada y salida)

Esta sección define explícitamente, por **nivel de prueba**, los **criterios de entrada** (entry criteria — qué debe estar listo para iniciar) y los **criterios de salida** (exit criteria — condiciones medibles para dar el nivel por concluido). Los criterios se identifican con prefijos EC-IN-<NIVEL>-NNN (entrada) y EC-OUT-<NIVEL>-NNN (salida) para poder ser referenciados desde reportes de ejecución y desde la matriz de trazabilidad.

13.1 Criterios de entrada (entry criteria)

Antes de iniciar cada nivel de prueba se deben cumplir las siguientes condiciones. Si falta alguna, el nivel **no arranca** y se documenta el bloqueo en el reporte diario de QA.

Nivel	ID	Condición de entrada
TU (unitarias)	EC-IN-TU-001	Código de la unidad mergeado en la rama del PR con build verde local
TU	EC-IN-TU-002	Dependencias de dobles de prueba (mocks, stubs, fakes) disponibles
TU	EC-IN-TU-003	Ambiente ENT-001 o ENT-002 operativo
TI (integración)	EC-IN-TI-001	Suite TU del módulo correspondiente en verde
TI	EC-IN-TI-002	Esquema de base de datos migrado a la versión bajo prueba
TI	EC-IN-TI-003	Especificación OpenAPI publicada y consistente con el código
TI	EC-IN-TI-004	LocalStack o testcontainers (S3, SQS) corriendo en ENT-002 ; o servicios AWS reales en ENT-003
TE2E	EC-IN-TE2E-001	Suite TI P0 en verde en ENT-003
TE2E	EC-IN-TE2E-002	Dataset de seed nightly cargado en ENT-003 (>= 1k tickets variados por estado/severidad)
TE2E	EC-IN-TE2E-003	Sink de email (Mailpit) y canal Slack de QA disponibles
TE2E	EC-IN-TE2E-004	Credenciales de prueba para los 3 roles (Reportante, Agente, Administrador) emitidas por Cognito dev
TUAT	EC-IN-TUAT-001	Suite TE2E P0 en verde en ENT-004 staging

Nivel	ID	Condición de entrada
TUAT	EC-IN-TUAT-002	Profesor/cliente notificado con al menos 48 h de anticipación
TUAT	EC-IN-TUAT-003	Documentación viva (.feature compilados) publicada en docs/uat/
TUAT	EC-IN-TUAT-004	Acceso de lectura al ambiente ENT-004 validado para el profesor/cliente
TSEC	EC-IN-TSEC-001	Build de la imagen del contenedor del SHA bajo prueba publicada en ECR
TSEC	EC-IN-TSEC-002	Suite TI completa en verde (evita falsos hallazgos sobre código roto)
TSEC	EC-IN-TSEC-003	Reglas de WAF y headers de seguridad desplegados en ENT-004
TSEC	EC-IN-TSEC-004	Ventana de pruebas invasivas (ZAP active scan, sqlmap) acordada con el equipo
TC (carga)	EC-IN-TC-001	Suite TE2E P0 en verde y ENT-004 libre de otras pruebas concurrentes
TC	EC-IN-TC-002	Dataset realista cargado en RDS (>= 10k tickets para TC-003/TC-004)
TC	EC-IN-TC-003	Métricas de baseline (CPU, RAM, conexiones RDS, profundidad SQS) capturadas antes del ramp-up
TC	EC-IN-TC-004	Scripts k6 versionados en tests/load/ y revisados por al menos 1 miembro adicional
TS (estrés)	EC-IN-TS-001	Resultados de carga (TC) aprobados y archivados
TS	EC-IN-TS-002	HPA, Cluster Autoscaler y alarmas CloudWatch configuradas y verificadas
TS	EC-IN-TS-003	Plan de recuperación documentado (cómo abortar y restaurar ENT-004)
TUX	EC-IN-TUX-001	Los 8 mockups publicados en docs/mockups/ sin cambios pendientes
TUX	EC-IN-TUX-002	Al menos 2 evaluadores independientes designados y briefing entregado
TUX	EC-IN-TUX-003	Herramientas de accesibilidad (axe-core, Lighthouse, contrast checker) instaladas

13.2 Criterios de salida (exit criteria)

Un nivel de prueba se considera concluido cuando se cumplen **todos** los criterios de salida aplicables. Los criterios son medibles con umbrales numéricos y se reportan en el resumen de la fase.

Nivel	ID	Condición de salida
TU	EC-OUT-TU-001	100% de los casos unitarios P0/P1 verdes en CI
TU	EC-OUT-TU-002	Cobertura >= 80% líneas y >= 75% ramas en módulos /src/domain/* y /src/usecases/*
TU	EC-OUT-TU-003	100% de cobertura en funciones críticas: cálculo de prioridad, evaluación de SLA, evaluación de RBAC
TU	EC-OUT-TU-004	Tiempo total de ejecución de la suite < 60 s
TU	EC-OUT-TU-005	0 defectos abiertos de severidad Crítica/Alta
TI	EC-OUT-TI-001	100% de casos TI P0 verdes en ENT-003
TI	EC-OUT-TI-002	100% de casos TI P1 verdes o con defecto documentado y aceptado por el equipo
TI	EC-OUT-TI-003	Latencia P95 por endpoint en condiciones nominales < 500 ms (excepto reportes CU-07, < 3 s)
TI	EC-OUT-TI-004	0 errores 5xx no esperados durante 3 ejecuciones consecutivas
TI	EC-OUT-TI-005	Reporte de cobertura de contract tests publicado (% de endpoints OpenAPI cubiertos >= 90%)
TE2E	EC-OUT-TE2E-001	Los 8 flujos E2E P0 verdes en ENT-004 durante 3 ejecuciones consecutivas
TE2E	EC-OUT-TE2E-002	Tiempo total de la suite E2E < 15 min en CI
TE2E	EC-OUT-TE2E-003	Evidencias (screenshots, video Playwright, logs estructurados) archivadas en docs/qa/e2e/
TUAT	EC-OUT-TUAT-001	Los 8 escenarios Gherkin principales firmados por profesor/cliente
TUAT	EC-OUT-TUAT-002	Defectos derivados de UAT clasificados (Crítica/Alta/Media/Baja) y priorizados antes del cierre
TUAT	EC-OUT-TUAT-003	Acta de aceptación archivada en docs/qa/uat/ con fecha, asistentes y firmas
TSEC	EC-OUT-TSEC-001	0 vulnerabilidades Críticas y 0 Altas abiertas en ZAP active scan

Nivel	ID	Condición de salida
TSEC	EC-OUT-TSEC-002	0 vulnerabilidades High/Critical conocidas en la imagen del contenedor (Trivy)
TSEC	EC-OUT-TSEC-003	0 hallazgos de secretos en repositorio (gitleaks)
TSEC	EC-OUT-TSEC-004	RBAC verificado para los 3 roles en el 100% de endpoints
TSEC	EC-OUT-TSEC-005	Reporte ZAP firmado y archivado en docs/qa/security/
TC	EC-OUT-TC-001	P95 < 2000 ms en GET /tickets con 10k registros (SLA CU-06)
TC	EC-OUT-TC-002	P95 < 500 ms en POST /tickets y PATCH /tickets/{id}/state en carga nominal
TC	EC-OUT-TC-003	Error rate < 1% en carga pico (TC-002, TC-006)
TC	EC-OUT-TC-004	RDS: CPU < 80% y Connections < 80% del límite durante toda la prueba
TC	EC-OUT-TC-005	Reporte k6 con percentiles (P50/P95/P99), throughput y error rate archivado por endpoint
TS	EC-OUT-TS-001	El sistema no entra en cascada de fallos en ninguno de los escenarios TS-001 a TS-008
TS	EC-OUT-TS-002	Recuperación del estado nominal en < 5 min tras retirar la sobrecarga
TS	EC-OUT-TS-003	0 mensajes SQS perdidos bajo saturación con escalado horizontal
TS	EC-OUT-TS-004	Alarmas CloudWatch declaradas en §5 Observabilidad de E1 dispararon correctamente en cada escenario
TUX	EC-OUT-TUX-001	0 violaciones bloqueantes de heurísticas Nielsen (severidad 4 — catastrófica)
TUX	EC-OUT-TUX-002	0 incumplimientos críticos de WCAG 2.1 AA en los 8 mockups
TUX	EC-OUT-TUX-003	Evaluación firmada por al menos 2 evaluadores independientes archivada en docs/qa/ux/

13.3 Criterios de aceptación globales del release (Definition of Done del producto)

Adicional al cumplimiento de los criterios por nivel (§13.1 y §13.2), el producto está listo para entregarse al cliente cuando se cumplan **todos** los siguientes criterios globales:

1. **Trazabilidad:** cada CU (CU-01 a CU-08) tiene al menos un caso de prueba ejecutado y aprobado por cada tipo aplicable en la matriz §12.1.
2. **% de SLA cumplido en pruebas de escalamiento:** >= 99% de los tickets críticos elegibles escalan dentro del minuto siguiente al vencimiento.
3. **Documentación:** reportes de ejecución archivados en docs/qa/ por cada tipo de prueba con fecha y commit hash.
4. **Defectos abiertos:** 0 defectos Críticos y 0 Altos abiertos al momento del cierre del release.
5. **Rollback verificado:** existe procedimiento documentado y probado de rollback del último despliegue.
6. **Acta de UAT firmada** por profesor/cliente.
7. **Observabilidad operativa:** dashboards de CloudWatch + alarmas activas y validadas en ENT-005 .

14. Riesgos de pruebas y mitigaciones

ID	Riesgo	Prob.	Impacto	Mitigación
R-01	El IdP concreto no está decidido (pregunta abierta §6 E1) — bloquea pruebas de JWT realistas	Alta	Alto	Usar un IdP mock conforme a OIDC durante D2-D4; reemplazar por el real en D5
R-02	El motor de email (SES/SendGrid/SMTP) no está decidido — bloquea pruebas de notificación E2E	Alta	Medio	Usar Mailpit/Mailhog como sink durante todas las fases; sustituir por el real en staging post-D4
R-03	Mecanismo del job de escalamiento sin decidir (ECS Scheduled Task / cron / EventBridge)	Media	Medio	Diseñar el test del job de forma agnóstica al disparador; invocar el handler directamente
R-04	Datos sintéticos no representativos del volumen real	Media	Alto	Sembrar dataset de 10k-50k tickets antes de TC-003/TC-004; documentar la distribución por categoría/severidad

ID	Riesgo	Prob.	Impacto	Mitigación
R-05	Ambiente staging compartido — tests pisan datos	Alta	Medio	Aislar por tenant /prefijo de TKT por ejecución; limpieza post-ejecución; ventanas dedicadas para carga/estrés
R-06	Concurrencia (optimistic vs pesimista) aún no resuelta — pruebas pueden quedar desalineadas	Alta	Medio	Documentar el supuesto (optimistic con version); recablear si la decisión cambia en D2
R-07	Slack workspace del equipo no permite bots de prueba	Media	Bajo	Usar canal dedicado de QA con webhook propio; alternativa: Discord en pruebas internas
R-08	Costos de AWS por carga/estrés sostenidos	Media	Medio	Limitar ventanas de pruebas de estrés a < 1 h; usar instancias t3 para staging
R-09	Tiempo del equipo (2 personas) ajustado para todos los tipos de prueba	Alta	Alto	Priorizar por riesgo: unitarias + integración + seguridad + UAT como P0; carga/estrés/UX en ventanas de demo
R-10	Drift entre mockups y backend real	Media	Bajo	Revisar mockups al inicio de cada entrega; documentar divergencias
R-11	Falsos positivos en escaneo de seguridad (ZAP)	Media	Bajo	Triage de cada hallazgo; mantener whitelist documentada
R-12	Datos sensibles reales en logs de staging	Baja	Alto	Ningún dato productivo en staging; anonimización de cualquier dataset importado

15. Calendario y secuencia de ejecución

El calendario se alinea con las entregas D2–D5 mencionadas en §6 de E1. Las fechas concretas dependen del cronograma final del curso y se confirmarán en el sprint planning del equipo. Esta sección presenta primero una **tabla maestra de fases** (§15.0), seguida del mapeo a entregas (§15.1), un **Gantt en ASCII** (§15.2) y la **cadencia continua** del pipeline (§15.3).

15.0 Tabla maestra de fases (cronograma por fase × entrega × semana)

Cada fase tiene un ID `CRON-F<n>` para ser referenciada desde los reportes semanales de avance. Las "semanas relativas" son contadas desde el inicio de la entrega D2 (semana 1 del cronograma de QA).

ID	Fase	Entrega(s)	Semanas relativas	Duración	Hito de cierre	Dependencias	Responsable principal
CRON-F1	Preparación de QA (plan, scaffolding de tests, ambientes)	Pre-D2	S0	1 sem	Plan aprobado + ENT-001 y ENT-002 operativos	E1 cerrado	Erick
CRON-F2	Pruebas unitarias (TU) — dominio core	D2	S1–S2	2 sem	Cobertura >= 80% líneas + EC-OUT-TU-* cumplidos	CRON-F1	Quien implementa cada unidad
CRON-F3	Pruebas de integración (TI) — endpoints core	D2	S2–S3	2 sem	TI P0 verde en ENT-003 + EC-OUT-TI-*	CRON-F2	Luis
CRON-F4	Pruebas de integración (TI) — adjuntos y presign	D3	S4	1 sem	TI-080/081/082 verdes	CRON-F3 + S3 desplegado	Luis
CRON-F5	Pruebas de integración asíncrona (TI-100 a TI-104)	D4	S5–S6	2 sem	Worker + job de escalamiento integrados y verdes	CRON-F4 + SQS/SES desplegados	Luis
CRON-F6	Pruebas E2E (TE2E-001 a TE2E-010)	D4	S6–S7	2 sem	TE2E P0 verde 3x consecutivas en ENT-004	CRON-F5 + ENT-004 listo	Erick

ID	Fase	Entrega(s)	Semanas relativas	Duración	Hito de cierre	Dependencias	Responsable principal
CRON-F7	Pruebas de seguridad (TSEC) — RBAC, JWT, ZAP, Trivy	D5	S7–S8	2 sem	0 vulnerabilidades Críticas/Altas	CRON-F6 + WAF + Cognito en ENT-004	Luis
CRON-F8	Pruebas de carga (TC-001 a TC-008)	D5	S8	1 sem	EC-OUT-TC-* cumplidos	CRON-F6 + dataset 10k tickets	Erick
CRON-F9	Pruebas de estrés (TS-001 a TS-008)	D5	S9	1 sem	EC-OUT-TS-* cumplidos	CRON-F8 aprobada	Erick
CRON-F10	Pruebas UX (TUX-001 a TUX-015) sobre mockups	D5	S8–S9	2 sem	Firma de 2 evaluadores	Mockups publicados	Luis
CRON-F11	Regresión consolidada + UAT (TUAT-001 a TUAT-008)	Demo	S10	1 sem	8 acta UAT firmadas	CRON-F6 a CRON-F10	Erick + Luis
CRON-F12	Cierre: reporte consolidado, lecciones aprendidas, archivo	Demo	S10	0.5 sem	Reporte final entregado al profesor	CRON-F11	Erick

15.1 Mapeo a entregas D2–D5

Entrega	Fecha objetivo	Foco de QA	Fases activas	Tipos de prueba activos
D2 — Cómputo + BD	Final de junio 2026	Endpoints core y persistencia	CRON-F2, CRON-F3	TU, TI (POST/GET/PATCH), TUAT CU-01 a CU-03
D3 — Almacenamiento + Red	Mediados de julio 2026	Adjuntos en S3, URLs prefirmadas, VPC	CRON-F4	TU, TI (presign), TSEC (TSEC-009/010/014/021), TUAT CU-05
D4 — Asíncrono + CD	Final de julio 2026	SQS, worker, job de escalamiento, notificaciones	CRON-F5, CRON-F6	TI asíncrona (TI-100 a TI-104), TE2E, TUAT CU-04, primeras TC (TC-001/TC-003)
D5 — Seguridad + Observabilidad	Mediados de agosto 2026	RBAC, JWT, headers, hardening, métricas	CRON-F7, CRON-F8, CRON-F9, CRON-F10	TSEC completa, TC completa, TS, TUX, TUAT consolidada
Demo final	Final de agosto 2026	Aceptación con profesor/cliente	CRON-F11, CRON-F12	TUAT firmada por CU; reporte consolidado

15.2 Gantt en ASCII (semanas relativas S0–S10)

Fase	S0	S1	S2	S3	S4	S5	S6	S7	S8	S9	S10
CRON-F1	[##]										
CRON-F2		[#####]									
CRON-F3			[#####]								
CRON-F4				[####]							
CRON-F5					[#####]						
CRON-F6						[#####]					
CRON-F7							[#####]				
CRON-F8								[####]			
CRON-F9									[####]		
CRON-F10									[#####]		
CRON-F11										[####]	
CRON-F12											[##]
Hitos:		^D2			^D3			^D4		^D5	^Demo

15.3 Cadencia continua

- Cada PR: TU + lint + Trivy + gitleaks + terraform fmt / validate / tflint / tfsec / checkov .

- Cada merge a `main` : TI + contract tests en `ENT-003` .
- Diario (nightly en `ENT-004`) : TE2E + un subset de carga (`TC-001` , `TC-003`).
- Semanal: ZAP active scan en `ENT-004` .
- Antes de cada entrega: suite completa por tipo asignado a esa entrega + TUAT del CU correspondiente.

16. Roles y responsabilidades

Equipo de 2 personas: **Luis André Morales** y **Erick Estuardo Saban**. Las responsabilidades se reparten para garantizar cobertura sin solapamiento innecesario.

Actividad / Tipo de prueba	Responsable principal	Apoyo	Aprobador
Plan de pruebas (este documento)	Erick	Luis	Profesor
Pruebas unitarias del dominio	Quien implementa la unidad	El otro miembro (PR review)	Equipo
Pruebas de integración API	Luis	Erick	Equipo
Pruebas E2E	Erick	Luis	Equipo
UAT y validación con cliente	Erick + Luis (juntos)	—	Profesor
Pruebas de seguridad	Luis	Erick	Equipo
Pruebas de carga y estrés	Erick	Luis	Equipo
Pruebas UX sobre mockups	Luis (evaluador 1)	Erick (evaluador 2)	Equipo
Reporte consolidado de QA	Erick	Luis	Profesor

16.1 Matriz RACI resumida

Actividad	Luis	Erick	Profesor
Diseño del plan	C	R	A
Ejecución pruebas técnicas (U//E2E/Carga/Estrés)	R	R	—
Ejecución UAT	R	R	A
Aprobación final	I	I	A

R = Responsable, A = Accountable, C = Consulted, I = Informed.

17. Entregables del plan de pruebas

Esta sección lista los **artefactos producibles** que este plan se compromete a entregar a lo largo del ciclo D2–D5. Cada entregable lleva un ID `ENT-DEL-NNN` , un responsable, un formato y un momento de entrega. Todos los entregables se versionan en el repositorio bajo `docs/qa/` salvo indicación contraria.

ID	Entregable	Descripción	Responsable	Formato	Frecuencia / Momento de entrega
ENT-DEL-001	Documento del plan de pruebas	Este documento <code>E2_PlanDePruebas.md</code> con todas sus secciones y anexos	Erick	Markdown versionado	Una vez, actualizado por entrega
ENT-DEL-002	Casos de prueba unitarios (TU)	Archivos <code>*.spec.ts</code> / <code>*.test.py</code> con los TU-001 a TU-018 implementados	Quien implementa la unidad	Código fuente en <code>tests/unit/</code>	Continuo, por PR

ID	Entregable	Descripción	Responsable	Formato	Frecuencia / Momento de entrega
ENT-DEL-003	Casos de prueba de integración (TI)	Suite TI-001 a TI-104 implementada con Supertest + testcontainers	Luis	Código fuente en <code>tests/integration/</code>	Continuo, por PR
ENT-DEL-004	Casos de prueba E2E (TE2E)	Suite TE2E-001 a TE2E-010 implementada con Playwright/Newman	Erick	Código fuente en <code>tests/e2e/</code>	D4
ENT-DEL-005	Escenarios Gherkin de UAT (TUAT)	Archivos <code>.feature</code> para TUAT-001 a TUAT-008	Erick + Luis	<code>.feature</code> en <code>tests/uat/</code> y documentación viva en <code>docs/uat/</code>	D5
ENT-DEL-006	Scripts de carga k6 (TC)	Scripts JS de k6 para TC-001 a TC-008 con sus thresholds	Erick	Código fuente en <code>tests/load/</code>	D5
ENT-DEL-007	Scripts de estrés (TS)	Scripts de saturación + plan de fault injection para TS-001 a TS-008	Erick	Código fuente en <code>tests/stress/</code> + <code>playbook md</code>	D5
ENT-DEL-008	Evaluación heurística + accesibilidad (TUX)	Plantilla de evaluación llenada por 2 evaluadores para TUX-001 a TUX-015	Luis	PDF firmado + capturas	D5
ENT-DEL-009	Reporte de ejecución TU	Salida JUnit XML + reporte HTML de cobertura	Quien implementa	JUnit XML + HTML	Por ejecución en CI
ENT-DEL-010	Reporte de cobertura unitaria	Reporte consolidado de cobertura con badge para README	Luis	HTML (<code>coverage/</code>) + badge SVG	Por merge a <code>main</code>
ENT-DEL-011	Reporte de ejecución TI	JUnit XML + log estructurado con <code>request/response</code>	Luis	JUnit XML + JSONL	Por ejecución en CI
ENT-DEL-012	Reporte de ejecución TE2E	Reporte Playwright con screenshots y videos de fallos	Erick	HTML report + MP4/PNG	Por ejecución nightly
ENT-DEL-013	Reporte de seguridad ZAP	Active scan + passive scan; clasificado por severidad; whitelist documentada	Luis	HTML + JSON; firmado	D5 + semanal
ENT-DEL-014	Reporte de seguridad Trivy + gitleiks	Vulnerabilidades de imagen Docker y hallazgos de secretos en repo	Luis	JSON + SARIF	Por PR + D5
ENT-DEL-015	Reporte de carga (TC) con gráficas	Percentiles P50/P95/P99, throughput, error rate, CPU/conn RDS por escenario	Erick	PDF/HTML con gráficas k6 + screenshots CloudWatch	D5
ENT-DEL-016	Reporte de estrés (TS) con curva de degradación	Punto de quiebre, tiempo de recuperación, comportamiento de alarmas	Erick	PDF/HTML con timeline de eventos	D5
ENT-DEL-017	Matriz de trazabilidad ejecutada	Matriz §12 con marca de ejecutado/aprobado/fecha por celda	Erick	Markdown + CSV	Por entrega
ENT-DEL-018	Acta de UAT firmada	Documento de aceptación por CU firmado por profesor/cliente	Erick + Luis	PDF firmado	Demo final

ID	Entregable	Descripción	Responsable	Formato	Frecuencia / Momento de entrega
ENT-DEL-019	Reporte consolidado de QA	Resumen ejecutivo del ciclo: defectos, cobertura, métricas, lecciones aprendidas	Erick	PDF + Markdown	Demo final
ENT-DEL-020	Lecciones aprendidas / postmortem	Retrospectiva del ciclo de QA con acciones para futuros cursos	Erick + Luis	Markdown en docs/qa/lessons-learned.md	Post-demo

Política de archivo y retención:

- Los reportes de ejecución (ENT-DEL-009 a ENT-DEL-016) se archivan en docs/qa/<tipo>/<fecha>--<sha>/ y se mantienen durante toda la vida del repositorio.
- Los reportes firmados (ENT-DEL-013, ENT-DEL-018) además se respaldan en el bucket S3 dedicado del equipo con object lock.
- Todo entregable referencia el commit hash sobre el que se ejecutó para garantizar reproducibilidad.

Anexo IA

Qué le pedimos a la IA

- Estructura del plan de pruebas siguiendo IEEE 829 / ISO/IEC/IEEE 29119 adaptado al alcance del curso.
- Borrador inicial de los casos de prueba por tipo (unitarias, integración, E2E, UAT, seguridad, carga, estrés, UX) a partir de los CUs y funcionalidades §4.x ya definidos en E1.
- Sugerencia de herramientas estándar por tipo de prueba consistentes con el stack AWS (ECS Fargate + RDS + S3 + SQS).
- Redacción de los escenarios Gherkin a partir de los criterios de éxito declarados en §3 de E1.
- Propuesta de matriz de trazabilidad CU × tipos de prueba y §4.x × tipos de prueba.
- Sugerencia de baseline de carga (RPS, VUs, percentiles) coherente con el tamaño de empresa declarado en E1 §1.
- Catálogo inicial de riesgos del proceso de QA y sus mitigaciones.

Qué aceptamos y editamos

- La estructura general (15 secciones + anexo) fue propuesta por la IA y aceptada tras alinearla con los entregables D2–D5 ya declarados en E1 §6 y con el formato del documento E1 existente (uso de IDs CU-XX, referencias §4.x, tablas markdown, tono profesional en español).
- Los casos de prueba fueron generados como borrador y **editados uno por uno** para que: (a) cada uno trazara a un CU o §4.x concreto, (b) el resultado esperado fuera verificable y no genérico, (c) los IDs siguieran un esquema legible (TU-, TI-, TE2E-, TUAT-, TSEC-, TC-, TS-, TUX-).
- Las herramientas sugeridas se filtraron al conjunto que el equipo conoce o puede aprender en el plazo del curso: Vitest/Jest/Pytest, Supertest, Testcontainers, Playwright/Cypress/Newman, k6/Locust/JMeter, OWASP ZAP, Burp, Trivy, gitleaks, axe-core, Lighthouse.
- Los escenarios Gherkin fueron ajustados para que cada Given/When/Then refleje literalmente el criterio de éxito declarado en §3 de E1 (por ejemplo: "número con formato TKT-XXXX", "menos de 2 segundos" en CU-06, "causa raíz y solución requeridas" en CU-03).
- La matriz de trazabilidad fue revisada **celda por celda** para confirmar que cada ☒ correspondiera a un caso de prueba real listado en las secciones 3–10 y no a un cubrimiento aspiracional.
- La sección de riesgos fue **extendida** por el equipo con riesgos derivados directamente de las preguntas abiertas de E1 §6: motor de email indefinido (R-02), IdP indefinido (R-01), mecanismo del job de escalamiento indefinido (R-03), concurrencia indefinida (R-06). Estos riesgos no estaban en el borrador inicial de la IA.
- Se ajustó la estructura del documento para alinearse **1:1 con la diapositiva oficial de la rúbrica del curso** (8 temas: objetivos, alcance, cronograma, tipos de prueba, entornos, riesgos, criterios de aceptación de entrada y salida, entregables). En particular se reescribió §13 separando explícitamente **entry criteria (EC-IN-*)** y **exit criteria (EC-OUT-*)** por nivel de prueba, se agregó la nueva sección §17 **Entregables del plan de pruebas** (ENT-DEL-001 a ENT-DEL-020) y se reforzaron §2.2 Ambientes (con IDs ENT-001 a ENT-005) y §15 Cronograma (con fases CRON-F1 a CRON-F12 y Gantt en ASCII) para hacer ambos temas explícitos y verificables.

Qué descartamos y por qué

- La IA propuso incluir pruebas de **compatibilidad multi-navegador** y **multi-dispositivo móvil** como sección propia. Lo descartamos porque el frontend productivo está fuera del scope del curso (E1 §6) y solo tenemos mockups low-fi; la compatibilidad se evaluará si y cuando el FE se construya.
- La IA sugirió pruebas de **migración de datos** desde un sistema previo. Lo descartamos porque no existe sistema previo declarado; el sistema arranca limpio.
- La IA propuso un esquema de **pruebas de regresión visual** con Percy/Chromatic. Lo descartamos para esta entrega porque el FE solo existe como mockups; se reconsiderará si en un curso posterior se implementa el FE.
- La IA propuso pruebas de **internacionalización** y **localización** (es-ES vs es-GT). Lo descartamos porque E1 §6 declara explícitamente que el sistema vive solo en español sin i18n.
- La IA sugirió un nivel de cobertura unitaria del 95%. Lo bajamos a 80% líneas / 75% ramas + 100% en módulos críticos, por realismo de un equipo de 2 personas con plazo del curso.
- La IA propuso pruebas dirigidas contra **PagerDuty** y **Datadog**. Lo descartamos porque están explícitamente fuera del scope en E1 §6.
- La IA propuso un esquema de pruebas con **mocks completos del DB** en integración. Lo descartamos a favor de **testcontainers/LocalStack** para evitar el clásico problema de mocks divergiendo de la realidad.
- La IA propuso un calendario con fechas absolutas concretas. Lo reemplazamos por una alineación con D2–D5 ya declaradas en E1 §6, sin comprometer fechas que aún no están confirmadas por el cronograma del curso.